

First program ChatGPT

- Name of two GenAI programs: **ChatGPT, Claude code**
- How you accessed them (e.g., Google query, CoPilot on my computer): **My openAI account web interface and anthropic account web interface**
- Prompt used. (You must use the same prompt for both GenAIs.)

prompt used:

make a ccp code named assignment3.cpp

The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

Here is a sets file format:

Command Description

EMAIL == The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume is one of the five strings listed above. Assume is a string which may contain spaces, but not commas Assume is in the format: MM-DD-YYYY

NEXT == Next email for the CEO to read. Display the information on the terminal in the following format:
Sender: Subject: Date:

READ == CEO has read email and dealt with it

COUNT == Display current count of unread emails

Importantly: The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

The output of this file must be:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read

d. Generated code from both GenAIs.

Chat GPT

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;
// stores one email
class Email {
public:
    string sender;
    string subject;
```

```

string date;

Email() {
    sender = "";
    subject = "";
    date = "";
}

Email(string s, string sub, string d) {
    sender = s;
    subject = sub;
    date = d;
}

int priority() const {
    if (sender == "Boss")
        return 5;
    if (sender == "Subordinate")
        return 4;
    if (sender == "Peer")
        return 3;
    if (sender == "ImportantPerson")
        return 2;
    return 1;
}

int dateValue() const {
    // Convert MM-DD-YYYY into YYYYMMDD so dates can be compared
    int month = stoi(date.substr(0, 2));
    int day = stoi(date.substr(3, 2));
    int year = stoi(date.substr(6, 4));

    return year * 10000 + month * 100 + day;
}

};

// MaxHeap implemented with a list-based vector
class MaxHeap {
private:
    vector<Email> heap;

    // Returns true when a should appear before b

```

```

bool higherPriority(const Email& a, const Email& b) const {
    if (a.priority() != b.priority())
        return a.priority() > b.priority();

    // Same sender category: newest email comes first
    return a.dateValue() > b.dateValue();
}

```

public:

```

MaxHeap() {
    heap.clear();
}

```

```

int size() const {
    return static_cast<int>(heap.size());
}

```

```

bool empty() const {
    return heap.empty();
}

```

```

void insert(const Email& email) {
    heap.push_back(email);

```

```

    int child = static_cast<int>(heap.size()) - 1;

```

```

    // Move the new email upward

```

```

    while (child > 0) {
        int parent = (child - 1) / 2;

```

```

        if (higherPriority(heap[child], heap[parent])) {
            Email temp = heap[child];
            heap[child] = heap[parent];
            heap[parent] = temp;
            child = parent;

```

```

        }

```

```

        else {
            break;

```

```

        }

```

```

    }

```

```

}

```

```

Email top() const {

```

```

        return heap[0];
    }

void removeMax() {
    if (heap.empty())
        return;

    // Replace the root with the last element
    heap[0] = heap.back();
    heap.pop_back();

    // Move the new root downward
    int parent = 0;

    while (true) {
        int left = 2 * parent + 1;
        int right = 2 * parent + 2;
        int largest = parent;

        if (left < static_cast<int>(heap.size()) &&
            higherPriority(heap[left], heap[largest])) {
            largest = left;
        }

        if (right < static_cast<int>(heap.size()) &&
            higherPriority(heap[right], heap[largest])) {
            largest = right;
        }

        if (largest == parent)
            break;

        Email temp = heap[parent];
        heap[parent] = heap[largest];
        heap[largest] = temp;

        parent = largest;
    }
}

};

int main() {

```

```

// The grader may redirect input from a test file.
// If an input filename is supplied as a command-line argument,
// that file is used instead.
ifstream inputFile;

// Read directly from standard input by default.
// This allows: Assignment3.exe < test.txt
istream& input = cin;

MaxHeap emailQueue;

string line;

while (getline(input, line)) {
    if (line.empty())
        continue;

    if (line.rfind("EMAIL ", 0) == 0) {
        string data = line.substr(6);

        string sender;
        string subject;
        string date;

        size_t firstComma = data.find(',');
        size_t secondComma = data.find(',', firstComma + 1);

        sender = data.substr(0, firstComma);
        subject = data.substr(firstComma + 1,
                               secondComma - firstComma - 1);
        date = data.substr(secondComma + 1);

        Email email(sender, subject, date);
        emailQueue.insert(email);
    }
    else if (line == "NEXT") {
        if (!emailQueue.empty()) {
            Email nextEmail = emailQueue.top();

            cout << "Next email:" << endl;
            cout << "Sender: " << nextEmail.sender << endl;
            cout << "Subject: " << nextEmail.subject << endl;
            cout << "Date: " << nextEmail.date << endl;

```

```

    }
}
else if (line == "READ") {
    if (!emailQueue.empty()) {
        emailQueue.removeMax();
    }
}
else if (line == "COUNT") {
    cout << "There are " << emailQueue.size()
        << " emails to read." << endl;
}
}

return 0;
}

```

Claude

```

// assignment3.cpp
// CEO email prioritizer using a hand-built, array(list)-based MaxHeap.
// Usage: ./assignment3 [testfile] (prompts for a filename if none is given)

```

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>

```

```

using namespace std;

```

```

// -----
// Email: one message in the CEO's inbox
// -----

```

```

class Email {
private:
    string sender;
    string subject;
    string date;    // MM-DD-YYYY (kept as given for display)
    int rank;       // higher = read sooner
    int dateKey;    // YYYYMMDD, larger = newer
    long order;     // arrival order, used only as a final tie-breaker

    int rankFor(const string& s) const {
        if (s == "Boss") return 5;
    }
}

```

```

    if (s == "Subordinate") return 4;
    if (s == "Peer") return 3;
    if (s == "ImportantPerson") return 2;
    return 1;    // OtherPerson (read last)
}

```

```

int keyFor(const string& d) const {
    // MM-DD-YYYY -> YYYYMMDD
    if (d.size() < 10) return 0;
    int mm = stoi(d.substr(0, 2));
    int dd = stoi(d.substr(3, 2));
    int yyyy = stoi(d.substr(6, 4));
    return yyyy * 10000 + mm * 100 + dd;
}

```

public:

```

Email() : sender(""), subject(""), date(""), rank(0), dateKey(0), order(0) {}

```

```

Email(const string& s, const string& subj, const string& d, long ord)
    : sender(s), subject(subj), date(d), order(ord) {
    rank = rankFor(sender);
    dateKey = keyFor(date);
}

```

```

string getSender() const { return sender; }
string getSubject() const { return subject; }
string getDate() const { return date; }

```

// True if this email should be read before 'other'

```

bool hasHigherPriorityThan(const Email& other) const {
    if (rank != other.rank) return rank > other.rank;    // category first
    if (dateKey != other.dateKey) return dateKey > other.dateKey; // newest first
    return order < other.order;                          // earlier arrival wins ties
}

```

```
};
```

// -----

// MaxHeap: list-based (dynamic array) binary max-heap of Emails

// -----

```
class MaxHeap {
```

```
private:
```

```

Email* data;

int count;

int capacity;

int parent(int i) const { return (i - 1) / 2; }

int left(int i) const { return 2 * i + 1; }

int right(int i) const { return 2 * i + 2; }

void swapAt(int a, int b) {
    Email tmp = data[a];
    data[a] = data[b];
    data[b] = tmp;
}

void grow() {
    int newCap = capacity * 2;
    Email* bigger = new Email[newCap];
    for (int i = 0; i < count; i++) bigger[i] = data[i];
    delete[] data;
    data = bigger;
    capacity = newCap;
}

void siftUp(int i) {
    while (i > 0 && data[i].hasHigherPriorityThan(data[parent(i)])) {
        swapAt(i, parent(i));
        i = parent(i);
    }
}

void siftDown(int i) {
    while (true) {
        int l = left(i), r = right(i), best = i;
        if (l < count && data[l].hasHigherPriorityThan(data[best])) best = l;
        if (r < count && data[r].hasHigherPriorityThan(data[best])) best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}

```

```

public:
    MaxHeap() : count(0), capacity(16) { data = new Email[capacity]; }
    ~MaxHeap() { delete[] data; }

    // Prevent accidental shallow copies of the internal array
    MaxHeap(const MaxHeap&) = delete;
    MaxHeap& operator=(const MaxHeap&) = delete;

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }

    void insert(const Email& e) {
        if (count == capacity) grow();
        data[count] = e;
        siftUp(count);
        count++;
    }

    // Returns the highest-priority email without removing it (caller checks isEmpty)
    const Email& peek() const { return data[0]; }

    // Removes the highest-priority email (caller checks isEmpty)
    void removeMax() {
        data[0] = data[count - 1];
        count--;
        if (count > 0) siftDown(0);
    }
};

// -----
// CEOInbox: wraps the heap and handles the four commands
// -----

class CEOInbox {
private:
    MaxHeap heap;
    long arrivals;

    static string trim(const string& s) {
        size_t a = s.find_first_not_of(" \t\r\n");
        if (a == string::npos) return "";
        size_t b = s.find_last_not_of(" \t\r\n");

```

```

        return s.substr(a, b - a + 1);
    }

```

public:

```

    CEOInbox() : arrivals(0) {}

```

```

    // args = "<sender category>,<subject line>,<date>"

```

```

    void addEmail(const string& args) {
        size_t c1 = args.find(',');
        size_t c2 = (c1 == string::npos) ? string::npos : args.find(',', c1 + 1);
        if (c1 == string::npos || c2 == string::npos) return; // malformed line, ignore

        string sender = trim(args.substr(0, c1));
        string subject = trim(args.substr(c1 + 1, c2 - c1 - 1));
        string date = trim(args.substr(c2 + 1));
        heap.insert(Email(sender, subject, date, arrivals++));
    }

```

```

    void next() const {
        if (heap.isEmpty()) {
            cout << "No emails to read." << endl;
            return;
        }
        const Email& e = heap.peek();
        cout << "Next email:" << endl;
        cout << "Sender: " << e.getSender() << endl;
        cout << "Subject: " << e.getSubject() << endl;
        cout << "Date: " << e.getDate() << endl;
    }

```

```

    void read() {
        if (!heap.isEmpty()) heap.removeMax();
    }

```

```

    void count() const {
        cout << "There are " << heap.size() << " emails to read." << endl;
    }

```

```

    // Parse and execute one line from the test file

```

```

    void processLine(const string& rawLine) {
        string line = trim(rawLine);
    }

```

```

        if (line.empty()) return;

        if (line.compare(0, 6, "EMAIL ") == 0) {
            addEmail(line.substr(6));
        } else if (line == "NEXT") {
            next();
        } else if (line == "READ") {
            read();
        } else if (line == "COUNT") {
            count();
        }
    }
}

};

// -----
int main(int argc, char* argv[]) {
    string filename;
    if (argc > 1) {
        filename = argv[1];
    } else {
        cout << "Enter test file name: ";
        getline(cin, filename);
    }

    ifstream in(filename.c_str());
    if (!in) {
        cerr << "Error: could not open file " << filename << " " << endl;
        return 1;
    }

    CEOInbox inbox;
    string line;
    while (getline(in, line)) {
        inbox.processLine(line);
    }

    return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space

complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude:

Good: This program complies and runs as expected. It correctly implements a MaxHeap to read and store the emails. It handles most general inputs plus some additional

Bad: stoi function throws a non-num date this causes a silent error if the date is inputted incorrectly

Chatgpt:

Good: This program complies and runs as expected. It correctly implements a MaxHeap to read and store the emails. It handles all general inputs

Bad: assumes that every email is perfectly formatted where bad input could cause silent errors

Execution time:

Claude:

Good: insert and remove have a big O of $(\log n)$ this is the rate limiting step

Bad: best big O speed with heap implementation.

Chatgpt:

Good: insert and remove have a big O of $(\log n)$ this is the rate limiting step

Bad: best big O speed with heap implementation.

Space complexity:

Claude:

Good: the storage is big O of n and the heap sits in one array, has a grow function where it only allocates originally 16 slots but then grows

Bad: each email object stores 2 dates so that it is easy to compare, also the size never shrinks so if there are many reads the largest size is still stored in the array. Very minimal

Chatgpt:

Good: there is a big o of n and the allocation of the heap grows with the each layer of the heap

Bad: the size never shrinks

Maintainability:

Claude:

Good: it is well organized and the email has proity while maxheap has the mechanics and the CEO inbox can handle commands.

Bad: no comments only documented headers.

Chatgpt:

Good: : it is well organized and the email has priority while maxheap has the mechanics and the CEO inbox can handle commands.

Bad: no comments only documented headers.

f. Select one to use as the basis for this assignment and justify why you picked it.

These two are very similar, I choose the Claude code one because it computes each email priority and date once instead of on every comparison. It also builds the more data structures like the email maxheap and ceo-inbox so the work is distributed more obviously and better for correctness and maintainability.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: currently bad dates cause the program to crash because the stoi function throws a non-num date and nothing catches that so one bad line can end the run. I changed something in the Email class and then used ctype to ensure it is correct and catch if incorrect

Execution time: none the log n implementation runs at maximum speed

Space complexity: shrink the array after the read. Currently it creates space as needed but then that space is emptied the heap storage is never cleared up effectively. I have added a resize that detects space if under a capacity

Maintainability: I have added detailed comments for every line

3. Make the changes identified in 2g above.

Added line by line comments on code and made changes to the Claude code with issues identified above

4. Compile and debug your program as described in the "C Basics" lecture.

5. Push both your source code, final cycle server executable file, and PDF of your

GenAI analysis to your GitHub repository. https://github.com/CarterGary-17/Fall2026_EECS348_Assignment_1